

EVALUATION 1 VS EVALUATION 2

Generated: 2026-04-24 16:35:51 UTC

GOAL

This report compares the first and second evaluation rounds of the IPPL RAG system. The aim is to explain not only the score change, but also what changed in the question set and what retrieval/prompting work we did between the two runs.

RUNS COMPARED

Run	Answer file	Question set	Host	Job id	Questions	Correct	Partial	Incorrect	Avg score
---	---	---	---	---	---	---	---	---	---
Evaluation 1	docs/evaluations/answers/eval_20260423T141904Z.json	docs/evaluations/eval_questions_v1.json	merlin-g-100.psi.ch	352636	113	45	24	44	0.504
Evaluation 2	docs/evaluations/answers/eval_v2_20260424T140027Z.json	docs/evaluations/eval_questions_v2.json	merlin-g-100.psi.ch	352660	100	44	24	32	0.560

HIGH-LEVEL COMPARISON

- The average score increased from 0.504 to 0.560 (+0.056).
- The raw question count dropped from 113 to 100 because Evaluation 2 intentionally removed questions that depended on tests, mini-apps, and example source files.
- Correct answers moved from 45 to 44. That looks almost flat in absolute count, but the error count dropped substantially because the question set became more on-scope for the current retrieval setup.
- This is not a pure apples-to-apples benchmark because Evaluation 2 changed the question set. It is better read as: after focusing the benchmark on src/ plus documentation, how well does the system now perform?

WHAT CHANGED IN THE QUESTION SET

Evaluation 2 removed 13 questions from the original 113-question set.

Removed Category	Count
---	---
examples_and_miniapps	7
numerical_meaning	5
testing_and_workflow	1

The removed questions were concentrated in the areas that were least aligned with the stated V2 scope:

REMOVED: EXAMPLES_AND_MINIAPPS

- ex_001 ? Which files make up the LandauDamping mini-app and what is the role of each?
- ex_002 ? What does AlpineManager.h provide to the alpine examples?
- ex_003 ? What is the difference between LandauDamping.cpp and LandauDampingMixedPrecision.cpp?
- ex_004 ? How does LandauDampingCorrectness.cpp validate results against FieldLandau_valid_result.csv?
- ex_005 ? How does the PenningTrap example configure the particle container and field solver?
- ex_006 ? What command-line arguments does the BumponTailInstability example accept?
- ex_007 ? Where are the HelloWorld, BasicsFields, BasicsParticles, and BasicsFFT example files located?

REMOVED: NUMERICAL_MEANING

- num_003 ? What does the Landau damping example simulate physically?
- num_004 ? What plasma phenomenon does the bump-on-tail instability example demonstrate?
- num_005 ? What does the PenningTrap example simulate, and what forces act on the particles?

- num_006 ? What does the cosmology StructureFormation mini-app model?
- num_008 ? What is the meaning of the load balancing threshold (lbthres) in the alpine mini-apps?

REMOVED: TESTING_AND_WORKFLOW

- test_001 ? How are IPPL unit tests organized and what do they parameterize over?

The core idea of V2 was to keep only questions that should be answerable from IPPL src/ code or repository documentation, and to drop questions whose best evidence lived in tests, mini-apps, or example files outside that scope.

WHAT WE WORKED ON BETWEEN THE TWO EVALUATIONS

LOCATION AND LIFECYCLE QUESTIONS

We worked on failures like ?Where is Kokkos initialized and finalized inside IPPL?? and related exact-location questions. The main change was to stop relying purely on semantic similarity and instead rescue exact API call sites directly.

ALGORITHM-SPECIFIC SOLVER QUESTIONS

We focused on questions like the FFT-based periodic/open-boundary Poisson solver prompts, where retrieval was drifting into generic FFT infrastructure instead of solver implementations.

DATA-FLOW QUESTIONS

We spent time on grid-to-particle and particle-to-grid handoff questions. The main theme was that these require cross-module evidence rather than more chunks from a single solver file.

COMPARISON QUESTIONS

We worked specifically on the FFT-vs-CG Poisson comparison case to make retrieval cover both solver families instead of flooding the prompt with FFT-only material.

PROMPT HYGIENE FOR SUPPLEMENTARY CONTEXT

We tightened how supplementary symbol-level chunks are rendered so they contribute only the most relevant support facts and a small code snippet, rather than a full second explanation block.

IMPLEMENTATION CHANGES

The most relevant changes between the two evaluations were:

- 0db9dba ? Created eval_job.sh so the evaluation can run non-interactively on gwendolen and write answer files under docs/evaluations/answers.
- 31a61e5 ? Improved lifecycle/location retrieval by detecting location intent, synthesizing exact API-bearing terms such as Kokkos::initialize, injecting literal matches, and boosting those exact call sites in reranking.
- cb0b888 ? Added comparison-aware retrieval for Poisson-solver tradeoff questions so the system retrieves both sides of the comparison instead of collapsing onto generic FFT infrastructure.
- 5511d34 ? Created the V2 evaluation question set and updated eval_job.sh so SSH runs use the filtered V2 questions and write V2-labelled answer files.
- 2e51a05 ? Shortened the content of supplementary symbol-entity chunks sent to the answering LLM so secondary context acts like support evidence rather than noisy full chunks.
- c07cd94 ? Added an eval-job-specific answer-model override mechanism so the cluster evaluation can use a different answer model without changing the normal local runtime settings.

CATEGORY-BY-CATEGORY COMPARISON

Category	Eval 1 Avg	Eval 2 Avg	Notes
---	---	---	---
File Purpose	0.923	0.923	Stable strength across both rounds.

Definition Location | 0.367 | 0.367 | Still weak; this is exactly where we focused the location/lifecycle retrieval work.
Class Responsibility | 0.885 | 0.885 | Stable strength across both rounds.
Algorithm | 0.417 | 0.417 | Still mixed; algorithm questions remain sensitive to whether solver implementation files are surfaced.
Data Flow | 0.333 | 0.333 | Still one of the weakest areas; cross-module handoff retrieval remains a hard problem.
Api Usage | 0.583 | 0.583 | Broadly similar between the two rounds.
Parallelism And Kokkos | 0.556 | 0.556 | Broadly similar between the two rounds.
Boundary And Halo | 0.500 | 0.500 | Broadly similar between the two rounds.
Numerical Meaning | 0.125 | 0.333 | Improved partly because V2 removed several mini-app-heavy numerical questions.
Examples And Miniapps | 0.000 | removed | Entire category removed in V2 because it depends on mini-app/example files outside the new scope.
Build And Install | 0.300 | 0.300 | Still weak even in V2; docs retrieval remains brittle here.
Testing And Workflow | 0.600 | 0.500 | Broadly similar between the two rounds.

INTERPRETATION

Evaluation 2 is better than Evaluation 1 in one important sense: it measures the system on a question set that is more aligned with the intended retrieval scope. That makes the result easier to trust as a signal about the current code-and-doc RAG pipeline rather than as a mixture of code retrieval quality and off-scope example coverage.

At the same time, the comparison also shows that simply filtering the question set was not enough. The remaining weak areas are exactly the ones we investigated during development: precise implementation-location questions, algorithm questions that need the right solver file instead of a nearby helper, and data-flow questions that need evidence from more than one subsystem.

The strongest improvement in process quality is not a single score jump; it is that the work between the two runs made the benchmark more focused and made retrieval behavior more explicit and controllable. That gives a much better foundation for the next round of iteration.

CONCLUSION

In short: Evaluation 2 is a cleaner benchmark, a slightly stronger result numerically, and a much better reflection of the retrieval work we actually did. The main development themes between the runs were narrowing the benchmark to src/ plus docs, improving exact location retrieval, improving solver comparison retrieval, and reducing prompt noise from supplementary chunks.